CI-CD with Github Actions

What is CI/CD, really?

Let's forget computers for a second. Imagine you run a **restaurant kitchen**.

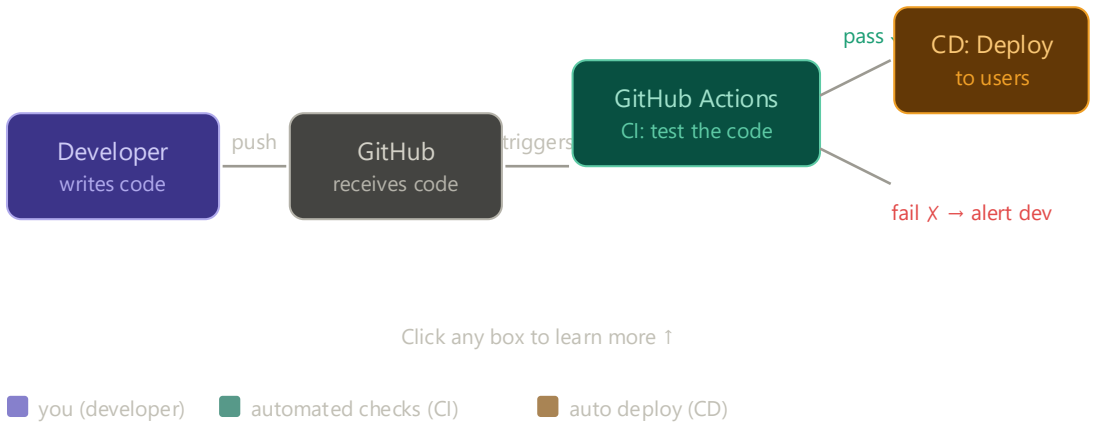
Every time a chef makes a new dish, someone needs to **check it** (does it taste right? is it safe to eat?) before it goes to the customer. And then someone needs to **deliver it** to the table.

In software:

- **CI (Continuous Integration)** = automatically *checking* every time a developer adds new code (like tasting the dish)
- **CD (Continuous Deployment/Delivery)** = automatically *delivering* that checked code to users (like sending it to the table)

Without CI/CD, developers manually check everything and manually deploy — which is slow, error-prone, and stressful. With CI/CD, a robot does it every time, instantly.

GitHub Actions is the robot. It lives inside GitHub and watches your code. When something happens (you push code, open a pull request), it wakes up and does jobs for you.



What is GitHub Actions

It listens for events like:

- push
- pull request

Then runs instructions you define

---

1. Workflow

👉 The full automation process

Example:
“Run tests when I push code”

2. Job

👉 A group of steps

Example:

- Install dependencies
- Run tests

3. Step

👉 A single action

Example:

- npm install
- npm test

4. Runner

👉 The computer that runs your code

GitHub gives:

- Ubuntu machine

- Fresh every time

🧱 Structure looks like this

Workflow
└── Job
    └── Steps

🗂️ Step 5 — Where we write CI/CD

Inside your project:

.github/workflows/main.yml

👉 YAML file (don't worry, it's just indentation-based config)

🧱 PHASE 1 — Build a SIMPLE Express App (from scratch)

No fancy stuff. Just something CI can test.

```
mkdir ci-cd-demo
cd ci-cd-demo
npm init -y
```

📦 Step 2 — Install basics

```
npm install express
npm install -D typescript ts-node-dev @types/node @types/express
```

⚙️ Step 3 — Setup TypeScript

```
npx tsc --init
```

Edit tsconfig.json (keep it simple):

🗂️ Step 4 — Folder structure

```
src/
  app.ts
```

🚀 Step 5 — Basic Express server

```
// src/app.ts

import express from "express";

const app = express();

app.get("/", (req, res) => {
  res.send("Hello CI/CD 🚀");
});

export default app;
```

▶️ Step 6 — Add start script

In package.json:

```
"scripts": {
  "dev": "ts-node-dev src/app.ts",
  "build": "tsc",
  "start": "node dist/app.js"
}
```

🧪 PHASE 2 — Add Testing (VERY IMPORTANT for CI)

CI without tests = useless

📦 Install Vitest

```
npm install -D vitest supertest
```

📁 Create test

src/app.test.ts

```
import request from "supertest";
import { describe, it, expect } from "vitest";
import app from "./app";

describe("GET /", () => {
  it("should return hello message", async () => {
    const res = await request(app).get("/");
    expect(res.text).toBe("Hello CI/CD 🚀");
  });
});
```

🧠 Add test script

```
"scripts": {
  "test": "vitest run"
}
```

✅ Run locally

```
npm test
```

👉 Make sure it passes BEFORE CI

⚠️ Reality Check

If your test fails locally…

👉 CI will fail too

CI is not magic — it just runs your code somewhere else.

🚀 PHASE 3 — Push to GitHub

```
git init
git add .
git commit -m "initial commit"
git branch -M main
git remote add origin <your-repo-url>
git push -u origin main
```

🤖 PHASE 4 — Your FIRST CI Pipeline

📁 **Create file**

.github/workflows/ci.yml

📜 **Add this:**

```
name: Basic CI Pipeline

on:
  push:
    branches: ["main"]

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node
        uses: actions/setup-node@v4
```

```
      with:
        node-version: 18

    - name: Install dependencies
      run: npm install

    - name: Run tests
      run: npm test
```

Explanation

◇ name: Basic CI Pipeline

This is just a label shown in GitHub Actions UI.

◇ **on:**

```
on:
  push:
    branches: ["main"]
```

Answer:
👉 “When I push code to main branch”

🧠 **Real meaning:**

This defines the **trigger event**

- push → runs when you push code
- branches: ["main"] → only for main branch

💡 Important thinking:

If you push to another branch like dev → ❌ this won't run

🔹 **jobs:**

jobs:

🧠 **Real meaning:**

A workflow can have multiple jobs (parallel or sequential)

Right now → you have **one job**

◇ build-and-test:

build-and-test:

🧠 **Real meaning:**

This is just a **job ID**

You could name it anything:

- backend-check
- ci-job

◇ runs-on: ubuntu-latest

runs-on: ubuntu-latest

Answer:
👉 "A fresh Linux computer"

🧠 **Real meaning:**

GitHub spins up a **virtual machine**

- OS: Ubuntu
- Clean every time
- Nothing installed except basics
- Every run is **fresh** → no memory → no cache

◇ steps:

steps:

🧠 **Real meaning:**

A job is made of steps executed in order

🔽 **Now the actual work starts**

🔹 Step 1 — Checkout Code

```
- name: Checkout code
  uses: actions/checkout@v4
```

🧠 **Real meaning:**

This step:

- Clones your repo into the runner machine

💡 Without this:
👉 There is NO CODE to run

🔸 Step 2 — Setup Node

```
- name: Setup Node
  uses: actions/setup-node@v4
  with:
    node-version: 18
```

🧠 **Real meaning:**

- Installs Node.js version 18
- Makes node, npm available

💡 Important thinking:

If you don’t specify version:
👉 You may get unexpected bugs

◇ Step 3 — Install dependencies

```
- name: Install dependencies
  run: npm install
```

🧠 **Real meaning:**

Reads package.json and installs:

- express
- vitest
- typescript
  etc.

💡 Without this:
👉 Your app cannot run

◇ Step 4 — Run tests

```
- name: Run tests
  run: npm test
```

Runs your test script:

"test": "vitest run"

💡 If tests fail:
👉 CI FAILS ❌
👉 You know something broke

🧠 What JUST happened (simple)

When you push code:

👉 GitHub:

1. Creates a fresh computer 🖥️
2. Downloads your code
3. Installs dependencies
4. Runs tests

🔍 PHASE 5 — Verify

Go to:

👉 GitHub → **Actions tab**

You'll see:

- ✅ Green → success
- ❌ Red → something broke

🧠 Important Thinking (Don't skip)

Ask yourself:

👉 "What happens if I break the code?"

🧪 Try this

Change your app:

```
res.send("Broken 🚨");
```

Run:

```
git add .
git commit -m "break test"
git push
```

👉 CI should FAIL ❌

💡 **This is the WHOLE POINT**

CI is not for success.

👉 It is for **catching failures automatically**

🧱 **PHASE 6 — Make it Slightly Better**

Let's add **build step**

Update workflow:

```
- name: Build project
  run: npm run build
```

🧠 **Now your pipeline does:**

1. Install
2. Test
3. Build

How the flow works..?

- You push code 📤
- Code is saved in repo ✅
- CI runs AFTER that 🤖

🧠 **Important Realization**

CI is **not stopping the push**

👉 It is only **checking the code after push**

🔥 **Then how do companies STOP bad code?**

This is the real missing piece 👇

✅ **Solution: Pull Requests + Branch Protection**

Instead of pushing directly to main, real teams do:

🪜 **Step 1 — Create a new branch**

git checkout -b feature/login

🪜 **Step 2 — Push branch**

git push origin feature/login

🪜 **Step 3 — Create Pull Request (PR)**

👉 "I want to merge this into main"

🪜 **Step 4 — CI runs on PR**

Now:

👉 If CI fails ❌
👉 You CANNOT merge into main

🪜 **Step 5 — Branch Protection (IMPORTANT)**

GitHub setting:

👉 "Require CI to pass before merging"

🧠 **Now behavior becomes:**

Situation	Result
Push to branch	Allowed
CI fails	❌ Cannot merge
CI passes	✅ Can merge

🎯 **Goal**

✓ Only allow merge if:

- CI passes ✅
- No broken code enters main 🚫

🚀 **Step-by-step setup**

1. Go to repo settings

- Open your repo
- Click **Settings**
- Click **Branches**

---

2. Add branch protection rule

- Click **“Add rule”**

---

3. Configure like this 👇

🔹 **Branch name pattern**

main

🔹 **Enable these options**

✅ **Require a pull request before merging**
→ No direct push to main

✅ **Require status checks to pass before merging**
👉 THIS is the key one

---

4. Select your CI check

After enabling status checks:

- GitHub will show your workflow name (e.g., build-and-test)
- Select it

👉 This connects your CI to merge rules

---

5. Optional but important (recommended)

✅ **Require branches to be up to date before merging**
→ Prevents outdated code merges

---

6. Save rule

Click **Create**

🧠 **What happens now**

Flow becomes:

You → create branch → push → open PR
↓
GitHub Actions runs CI
↓
✅ PASS → merge allowed
❌ FAIL → merge blocked

🔥 Real-world workflow (important)

You should now work like this:

```
git checkout -b feature/login
# make changes
git push origin feature/login
```

Then:
👉 Open Pull Request → NOT direct push to main

⚠️ Common beginner mistake

If you push directly to main, this protection:
❌ won't stop you (unless you disable direct pushes)

If you want strict mode:

👉 Enable:

Require pull request before merging
+ Restrict who can push

🧪 Test it (do this once)

1. Break your code intentionally
2. Push branch
3. Create PR

👉 You'll see:
🚫 "Merge blocked — checks failing"

⚡ Reality check

This is exactly what companies do:

- No one pushes to main

- Everything goes through CI
- Broken code NEVER reaches production

🧠 **First, reality check**

Right now you have:

👉 **CI (Continuous Integration)**

- Build ✅
- Test ✅

But **nothing is actually deployed**.

So your pipeline is basically:

Code → Test → Stop ❌

🚀 **What CD actually means**

CD = Continuous Deployment (or Delivery)

👉 After CI passes:

- Your app is **automatically deployed somewhere**

Code → Test → Deploy ✅

🔥 **Two types of CD (important distinction)**

1. Continuous Delivery

👉 Needs manual approval before deploy

2. Continuous Deployment (real automation)

👉 Auto deploy after CI passes (no human step)

1. A place to deploy

Examples:

- Render (easy, beginner-friendly)
- Railway
- AWS (advanced)
- Vercel (frontend mainly)

2. Secrets (very important)

API keys, tokens, etc.

In **GitHub**:
👉 Settings → Secrets → Actions

Example:

RENDER_API_KEY=xxxxx

3. Deploy step in workflow

Your current CI file probably ends at:

- run: npm test

Now you add **deploy job** 👇

deploy:
needs: build # 👈 runs only if build passes
runs-on: ubuntu-latest

steps:
- name: Deploy to Render
run: echo "Deploying app..."

🧠 **Key concept (don't miss this)**

needs: build

👉 This is your **CI → CD connection**

- If CI fails ❌ → deploy never runs
- If CI passes ✅ → deploy starts

⚡ **Real deployment (example idea)**

For **Render**, you usually:

- Connect GitHub repo
- Auto deploy on push

Add TypeScript Check

Why?

Right now:
👉 Your code can have type errors and still pass CI ❌

Add script in package.json

```
"scripts": {
  "type-check": "tsc --noEmit"
}
```

Update CI

```
- name: Type check
  run: npm run type-check
```

🧠 **Meaning:**

👉 “Check types, but don’t build files”

✅ **Step 2 — Add Linting**

Install ESLint

```
npm install -D eslint @typescript-eslint/parser @typescript-eslint/eslint-plugin
```

Create config (simple)

```
// eslint.config.js
export default [
  {
    files: ["**/*.ts"],
    languageOptions: {
      parser: "@typescript-eslint/parser",
```

```
  },
  plugins: {
    "@typescript-eslint": require("@typescript-eslint/eslint-plugin"),
  },
  rules: {
    semi: ["error", "always"],
  },
 },
];
```

Add script

```
"lint": "eslint ."
```

Add to CI

```
- name: Lint check
  run: npm run lint
```

Add Caching (REAL WORLD SKILL)

Without this:
👉 CI is slow → bad developer experience

Add this BEFORE install step:

```
- name: Cache dependencies
  uses: actions/cache@v4
  with:
    path: ~/.npm
    key: ${{ runner.os }}-node-${{ hashFiles('package-lock.json') }}
    restore-keys: |
      ${{ runner.os }}-node-
```

🧠 **Simple:**

👉 “If dependencies didn’t change, don’t reinstall everything”

Add PostgreSQL Service

Add this inside job:

```
services:
  postgres:
    image: postgres:15
    env:
      POSTGRES_USER: test
      POSTGRES_PASSWORD: test
      POSTGRES_DB: testdb
    ports:
      - 5432:5432
    options: >-
      --health-cmd="pg_isready"
      --health-interval=10s
      --health-timeout=5s
      --health-retries=5
```

👱 **Simple:**

👉 “Start a database for testing”

⚙️ **Step 5 — Add ENV for Prisma**

```
env:
  DATABASE_URL: postgres://test:test@localhost:5432/testdb
```

🧱 **Step 6 — Run Prisma in CI**

Add steps:

```
- name: Run migrations
  run: npx prisma migrate deploy

- name: Generate Prisma client
  run: npx prisma generate
```

🧪 Step 7 — Integration Tests

Now your tests will:

👉 Actually hit a real DB

🧠 These are used in almost EVERY pipeline

1️⃣ actions/checkout

👉 Get code

2️⃣ actions/setup-node

👉 Setup runtime

3️⃣ actions/cache

👉 Speed optimization

4️⃣ run

👉 Execute real work (MOST IMPORTANT)

5️⃣ Services (Docker)

👉 DB, Redis, etc.

🔥 1. What is CI/CD?

✅ **Simple Answer:**

CI/CD is a process that automates:

- testing code (CI)
- delivering/deploying code (CD)

💪 **Strong Answer (say this in interview):**

CI ensures every code change is automatically tested and validated. CD ensures that once the code passes all checks, it can be safely deployed to production.

🔥 2. Why do we need CI/CD?

❌ **Weak answer:**

“Automation”

✅ **Strong answer:**

It reduces human error, ensures consistent testing, catches bugs early, and makes collaboration in teams safer by validating every change before merging.

🔥 3. What happens when you push code?

✅ **Strong Answer:**

When I push code, the CI pipeline is triggered. It checks out the code, sets up the environment, installs dependencies, runs type checks, linting, tests, and build. If any step fails, the pipeline fails.

🔥 4. Does CI stop bad code from entering the repo?

❗ **Trick Question**

✅ **Correct Answer:**

No, CI runs after the code is pushed. To prevent bad code from reaching main, we use pull requests and branch protection rules that require CI to pass before merging.

🔥 5. What is the difference between CI and CD?

✅ **Answer:**

- CI → testing and validation
- CD → deployment

💪 **Better:**

CI focuses on integrating and validating code changes, while CD automates the release process either by preparing builds (delivery) or deploying them (deployment).

🔥 6. What steps would you include in a backend CI pipeline?

✅ **Strong Answer:**

Checkout code, setup runtime, install dependencies, run type checks, linting, unit tests, integration tests with database, and build the project.

🔥 7. What are GitHub Actions?

✅ **Answer:**

GitHub Actions is a CI/CD tool that allows us to automate workflows like testing and deployment based on events such as push or pull requests.

🔥 8. What is a workflow, job, and step?

✅ **Answer:**

- Workflow → full pipeline
- Job → group of steps
- Step → single task

💪 **Better:**

A workflow defines the automation, jobs are independent units of work, and steps are individual commands executed inside a job.

🔥 9. What is a runner?

✅ **Answer:**

A runner is the machine that executes the CI jobs. GitHub provides hosted runners like Ubuntu.

🔥 10. Why do we use caching in CI?

✅ **Answer:**

To speed up builds by avoiding reinstalling dependencies every time.

💪 **Better:**

Caching reduces CI execution time by reusing previously installed dependencies when the lock file hasn't changed.

🔥 11. What are integration tests and why important?

✅ **Strong Answer:**

Integration tests verify that different parts of the system work together, such as API and database. They are important because many real-world bugs occur due to interactions between components.

🔥 **12. How do you test database in CI?**

✅ **Answer:**

By spinning up a database service (like PostgreSQL) using Docker in the CI pipeline, setting environment variables, running migrations, and executing integration tests.

🔥 **13. Where do you store secrets?**

✅ **Answer:**

In GitHub Secrets, not in code.

🔥 **14. What happens if CI fails?**

✅ **Answer:**

The pipeline stops, and the code should not be merged until the issue is fixed.

🔥 **15. What is branch protection?**

✅ **Answer:**

It prevents merging code into main unless conditions like passing CI checks are met.

🔥 **16. How would you deploy using CI/CD?**

✅ **Answer:**

After CI passes, we can build a Docker image, push it to a registry, and deploy it to a server or cloud platform.

🔥 17. What are the most used GitHub Actions?

✅ **Answer:**

actions/checkout, actions/setup-node, actions/cache, and running shell commands are the most commonly used.

🔥 18. CI is failing, what do you do?

✅ **Strong Answer:**

I check the logs to identify the failing step, try to reproduce the issue locally, and fix the root cause instead of bypassing the failure.

🔥 19. Difference between unit and integration tests?

✅ **Answer:**

- Unit → test individual function
- Integration → test system together

Unit tests isolate logic, while integration tests verify real interactions like API with database.

🔥 20. What is the biggest advantage of CI/CD?

✅ **Best Answer:**

Confidence. It ensures that every change is validated automatically, reducing risk in production.